

Introduction to React

- **React** is a front-end **JavaScript library** for building dynamic and reusable UI components.
- Created by **Facebook (Meta)**.
- Follows **component-based architecture**.
- Uses **Virtual DOM** and **Declarative programming**.
- Enables **Single Page Applications (SPA)**.

Component	Independent, reusable UI block.
JSX	JavaScript XML for embedding HTML in JS.
Virtual DOM	React's internal representation of the real DOM.
State	Data that changes within a component.
Props	Input data passed from parent to child.
Hooks	Functions that let you use state & lifecycle in functional components.

`npx create-react-app react-demo`

`cd react-demo`

`npm start`

src/App.js:

```
function App() {  
  return <h1>Hello React World 🌍</h1>;  
}  
  
export default App;
```

1. What is React and why is it used?

2. What are the advantages of using React?
3. Explain SPA and how React helps build it.

2. JSX (JavaScript XML)

- Allows writing HTML elements in JavaScript.
- JSX → compiled to `React.createElement()` by Babel.
- JSX improves readability and debugging.

Rules

- Must return **one parent element**.
- Use `className` instead of `class`.
- JavaScript expressions must be enclosed in `{}`.

```
function Greeting() {  
  const name = "Divya";  
  return <h2>Welcome, {name}!</h2>;  
}
```

- What is JSX and why is it needed?
- Can React work without JSX?

3. Components in React

React uses **components** to break UI into independent blocks.

Types

1. **Functional Component** → Simple JS function returning JSX.
2. **Class Component** → ES6 class extending `React.Component`.

1. Functional Components (Modern React)

- Created using **JavaScript functions**.
- Can use **Hooks** for state and lifecycle.
- **Simple and preferred** in modern React.

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

```
}
```

OR (Arrow Function)

```
const Welcome = ({ name }) => <h1>Hello, {name}</h1>;
```

2. Class Components (Older React)

- Created using **ES6 classes**.
- Use **this.state** and **this.props**.
- Lifecycle handled by **methods** (like componentDidMount()).

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

Component Structure

Every React component typically includes:

1. **Import Statements**
2. **Function/Class Definition**
3. **JSX Return**
4. **Export**

Example:

```
import React from "react";  
function Header() {  
  return <h2>My Website Header</h2>;  
}  
export default Header;
```

4. Props and State

Props	Immutable data passed from parent to child.
--------------	---

State	Mutable data maintained inside a component.
--------------	---

Props (Properties)

- Used to **pass data** from **parent** → **child** component.
- **Read-only**, cannot be modified by the child.

```
function Greeting(props) {  
  return <p>Welcome, {props.name}</p>;  
}
```

Usage:

```
<Greeting name="Divya" />
```

State

- **Manages internal data** inside a component.
- Can be changed using **useState** (functional) or **this.setState** (class).

```
const [count, setCount] = useState(0);
```

Component Hierarchy

- Components can be **nested** inside other components.
- **Parent** → **Child** → **Sub-child** relationships form the **component tree**.

```
function App() {  
  return (  
    <div>  
      <Header />  
      <Content />  
      <Footer />  
    </div>  
  );  
}
```

Reusability

- Components can be reused anywhere in the app.
- Makes the app **modular, maintainable, and scalable**.

5. Lists and Keys

Lists

- Used to **render multiple elements** dynamically from arrays.
- Implemented using JavaScript's `.map()` method.

Example:

```
const names = ["Divya", "Sandhiya", "Meena"];
const list = names.map((n, i) => <li key={i}>{n}</li>);
```

Keys

- A **unique identifier** for each list item.
- Helps React **track changes efficiently** during re-render.
- Should be **unique and stable** (use id, not array index if data can change).

Example:

```
students.map(s => <li key={s.id}>{s.name}</li>);
```

Best Practices

- ✓ Always use a **unique key** prop.
- ✓ Avoid using **index as key** in dynamic lists.
- ✓ Keep list rendering **clean and modular** (use child components if complex).

Real-Time Example

```
function TodoList() {
  const [tasks, setTasks] = useState(["Read", "Write"]);
  return (
    <ul>
      {tasks.map((task, index) => <li key={index}>{task}</li>)}
    </ul>
  );
}
```

```
);  
}
```

6. Conditional Rendering in React JS

- **Conditional Rendering** means **showing or hiding UI elements** based on a certain condition (like if, else, or a state value).
- Works **just like conditional statements in JavaScript** — React decides **what to render** dynamically.

✅ To show different UI based on user interaction or state.

✅ Example use cases:

- Show login form if user not logged in
- Show success/error messages
- Toggle components (dark/light mode, etc.)

1. Using if Statement

```
function Greeting(props) {  
  if (props.isLoggedIn) {  
    return <h2>Welcome Back!</h2>;  
  } else {  
    return <h2>Please Login</h2>;  
  }  
}
```

Based on isLoggedIn, React chooses which JSX to render.

2. Using Ternary Operator (?:)

Short and most commonly used.

```
function UserStatus({ isOnline }) {  
  return (  
    <p>{isOnline ? "User is Online 🟢" : "User is Offline 🔴"}</p>  
  )  
}
```

```
);  
}
```

3. Using Logical AND (&&)

Best for rendering something *only if* a condition is true.

```
function Notification({ hasMessage }) {  
  return (  
    <div>  
      <h3>Inbox</h3>  
      {hasMessage && <p>You have new messages!</p>}  
    </div>  
  );  
}
```

If hasMessage is true → message appears; else → nothing renders.

4. Using switch Statement

Useful when there are **multiple conditions**.

```
function Status({ code }) {  
  switch (code) {  
    case 200: return <p>✅ Success</p>;  
    case 404: return <p>❌ Not Found</p>;  
    default: return <p>⚠️ Unknown Status</p>;  
  }  
}
```

7. Handling Events

- **Event Handling** in React means **responding to user actions** (like clicks, typing, form submit, etc.) using event handlers.

- React events are similar to DOM events but follow **camelCase naming** and use **JSX syntax**.

Key Points

- ✓ Event names use **camelCase** (e.g., onClick, onChange, onSubmit).
- ✓ Event handler is a **function**, not a string.
- ✓ React **automatically wraps events** in a **SyntheticEvent** (cross-browser wrapper for native events).
- ✓ Use **arrow functions** or bind methods for event handling.

1. Example: Click Event

```
function ClickExample() {  
  function handleClick() {  
    alert("Button Clicked!");  
  }  
  
  return <button onClick={handleClick}>Click Me</button>;  
}
```

Explanation:

- onClick triggers the function handleClick when the button is pressed.

2. Event Handler with Parameters

```
function GreetUser({ name }) {  
  const greet = (user) => alert(` Hello, ${user}!` );  
  return <button onClick={() => greet(name)}>Greet</button>;  
}
```

- Use an **arrow function** to pass arguments.

3. Using Arrow Functions Inline

```
<button onClick={() => alert("Inline Event!")}>Click</button>
```

Use sparingly — can affect performance if re-rendered frequently.

8. Hooks (Functional Components)

- Hooks introduced in React 16.8 to handle state and lifecycle logic inside functional components.

Common Hooks

Hook	Purpose
useState()	Manage component state
useEffect()	Side effects (API calls, timers)
useRef()	Access DOM elements
useContext()	Share data globally
useReducer()	Manage complex state logic

- **Hooks** are **functions** introduced in **React 16.8** that allow you to use **state**, **lifecycle methods**, and **other React features** inside **functional components** — without using class components.
- Hooks start with the word **“use”** (e.g., useState, useEffect).

Why Hooks?

Before hooks:

- State & lifecycle were available only in **class components**.

With hooks:

- Functional components can manage **state**, **side effects**, and **context** easily.

2. Rules of Hooks

1. Only Call Hooks at the Top Level

- Don't call them inside loops, conditions, or nested functions.

2. Only Call Hooks from React Functions

- Call them from React functional components or custom hooks only.

1. useState()

Adds local state to functional components.

Syntax: `const [state, setState] = useState(initialValue);`

Example: Counter

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </>
  );
}

export default Counter;
```

Key Points:

- State updates are **asynchronous**.
- Updating state triggers a **re-render**.

2. useEffect()

Handles **side effects** — API calls, DOM updates, or timers.

Syntax:

```
useEffect(() => {
  // Side effect code
  return () => {
    // Cleanup (optional)
  };
});
```

```
}, [dependencies]);
```

Example: Timer

```
import { useState, useEffect } from "react";

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => setSeconds(s => s + 1), 1000);
    return () => clearInterval(interval);
  }, []);

  return <h3>Time: {seconds}s</h3>;
}
```

Dependency Array Behavior

Dependency	Behavior
[]	Runs only once (on mount)
[state]	Runs when the state changes
(no array)	Runs on every render

3. useContext()

Allows data sharing across components without prop drilling.

Example:

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function App() {
  return (
```

```
<ThemeContext.Provider value="dark">
  <Child />
</ThemeContext.Provider>

);
}
```

```
function Child() {
  const theme = useContext(ThemeContext);
  return <h3>Theme: {theme}</h3>;
}
```

4. useRef()

- Access DOM elements directly.
- Store mutable values that persist without causing re-renders.

Example:

```
import { useRef } from "react";
```

```
function FocusInput() {
  const inputRef = useRef(null);
  const focusInput = () => inputRef.current.focus();

  return (
    <>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus Input</button>
    </>
  );
}
```

5. useMemo()

Memoizes (caches) the result of a calculation — improves performance.

Example:

```
import { useMemo, useState } from "react";

function ExpensiveCalc() {
  const [num, setNum] = useState(1);
  const [count, setCount] = useState(0);

  const square = useMemo(() => {
    console.log("Calculating...");
    return num * num;
  }, [num]);

  return (
    <>
      <p>Square: {square}</p>
      <button onClick={() => setNum(num + 1)}>Change Number</button>
      <button onClick={() => setCount(count + 1)}>Re-render</button>
    </>
  );
}
```

6. useCallback()

Memoizes **function definitions** to prevent unnecessary re-renders.

Example:

```
import { useState, useCallback } from "react";
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  const increment = useCallback(() => setCount(c => c + 1), []);  
  
  return (  
    <>  
      <h2>{count}</h2>  
      <button onClick={increment}>+</button>  
    </>  
  );  
}
```

7. useReducer()

Manages complex state logic — alternative to useState.

Example:

```
import { useReducer } from "react";  
  
const reducer = (state, action) => {  
  switch (action.type) {  
    case "inc": return { count: state.count + 1 };  
    case "dec": return { count: state.count - 1 };  
    default: return state;  
  }  
};  
  
function Counter() {  
  const [state, dispatch] = useReducer(reducer, { count: 0 });  
  return (  

```

```

<>
  <h2>{state.count}</h2>
  <button onClick={() => dispatch({ type: "inc" })}>+</button>
  <button onClick={() => dispatch({ type: "dec" })}>-</button>
</>
);
}

```

9. Routing with React Router

Routing enables page navigation without full reload.

npm install react-router-dom

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
```

```

function App() {
  return (
    <BrowserRouter>
      <Link to="/">Home</Link> | <Link to="/about">About</Link>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

```